GA4 DATA PIPELINE FOR PORTFOLIO-WIDE ANALYSIS

Internship Realisation

[2026.05.15]
[Version 4.0.0]
Author: Alex Hendrickx
Contact: alex.hendrickx@iodigital.com

warning Confidentiality Notice Company names referenced in this document have been anonymised unless explicitly attributed. The only client names disclosed are those that iO Digital has publicly acknowledged on its corporate website. All other accounts are identified by sector and anonymised identifier.

Abstract

This project addresses the challenge of siloed client data within a digital agency's extensive Google Analytics 4 (GA4) portfolio comprising approximately 600 properties across 463 accounts. The primary objective was to design and implement a scalable, robust data pipeline to enable portfolio-wide analysis of GA4 data. The implemented solution automates the extraction of data via the GA4 Data API [2], stores it in a local SQLite database (development) or Google BigQuery (production) [4], following an Extract, Load, Transform (ELT) architecture with a storage abstraction layer enabling seamless backend switching. Key processes include programmatic data enrichment with industry classifications using large language models (LLMs) [7] and a privacy-by-design approach where all published outputs aggregate to sector level, ensuring no individual client data is identifiable. The final result is a structured, sector-enriched dataset that enables the agency to identify cross-client trends, establish performance benchmarks across 17 industry sectors, and derive strategic insights. The pipeline's performance was validated against the full client portfolio and demonstrated linear scalability.

Keywords: Google Analytics 4, data pipeline, ELT architecture, BigQuery, sector benchmarking, AI classification, data privacy, portfolio analysis

Acknowledgements

I would like to extend my sincere gratitude to my mentors and colleagues at iO Digital for their invaluable guidance, support, and feedback throughout this internship project. Their expertise and encouragement were instrumental in the successful development and implementation of this data pipeline.

Table of Contents

This document is structured as follows:

Abstract 1

Acknowledgements 1

Table of Contents 1

1. Introduction 1

1.1. Context and Problem Statement 1

1.2. Project Objective and Proposed Solution 1

1.3. Scope and Delimitations 1

1.4. Document Structure 1

2. Theoretical Framework & Literature Review 1

2.1. Data Pipeline Architectures (ETL vs. ELT) 1

2.2. Cloud Data Warehousing with BigQuery 1

2.3. Google Analytics 4 Data Model 1

2.4. Data Enrichment and Classification Techniques 1

2.5. Data Anonymisation and Privacy Principles 1

3. Methodology: Designing the Solution 1

3.1. Overall Architectural Design 1

3.2. Data Extraction Strategy 1

3.3. Data Transformation and Loading 1

3.4. Data Enrichment Process 1

3.5. Data Classification Model 1

3.6. Data Anonymisation Protocol 1

3.7. Technology Stack Justification 1

4. Implementation and Results 1

4.1. Development of the Extraction Module 1

4.2. Implementation of the Data Warehouse Schema 1

4.3. Execution of Enrichment and Classification Scripts 1
4.4. Final Anonymised Dataset 1
4.5. Pipeline Performance and Scalability Tests 2
4.6. Pipeline Orchestration 2
4.7. Storage Abstraction Layer 2
4.8. Fact Table Architecture 2
4.9. Production Performance Metrics 3

5. Benchmark Validation and Data Quality Assurance 4
5.1. Validation Methodology 4
5.2. Concentration Analysis: High-Volume Accounts 4
5.3. Invalid Data Detection: Bot Traffic 5
5.4. Tracking Configuration Artifacts 6
5.5. Geographic Bias in Temporal Metrics 6
5.6. Content-Type Effects on Engagement Metrics 6
5.7. Reliability Classification Framework 6
5.8. Sector Override Mechanism 7
5.9. Deviation from Initial Design: Sector Taxonomy 9
5.10. Summary of Metric-Level Caveats 9

6. Analysis and Discussion 10
6.1. Answering the Research Question 10
6.2. Effectiveness of Portfolio-Wide Analysis 10
6.3. Challenges and Limitations 10
6.4. Critical Evaluation of the Chosen Methodology 10
6.5. Benchmark Methodology and Data Quality 10

7. Conclusion and Recommendations 10
7.1. Conclusion 10
7.2. Recommendations for Future Work 10
7.3. Recommendations for the Organisation 10
7.4. Recommendations Derived from Benchmark Validation 10

Bibliography 10
References 10

Appendix: Use of Generative AI Tools 10

1. Introduction

1.1. Context and Problem Statement

A modern digital agency manages a diverse portfolio of clients, each with its own digital footprint and associated analytics data. Within the organisation under study, iO Digital, this translates to approximately 600 individual Google Analytics 4 (GA4) properties across 463 client accounts. While each property offers a wealth of information about a single client's performance, this data exists in isolated silos. Consequently, the organisation faces a significant strategic challenge: the inability to perform cross-client analysis.

This fragmentation prevents the agency from utilising its unique position overlooking a vast and varied digital landscape. Valuable insights into sector-wide trends, market movements, and comparative performance benchmarks remain inaccessible within individual GA4 accounts. As a result, analysis and reporting are confined to a reactive, client-by-client basis. The potential to identify broader opportunities, proactively advise clients based on industry trends, or develop data-backed strategic insights at a portfolio level remains largely unrealised. This represents a significant untapped opportunity to transform a collection of disparate data points into a strategic asset.

1.2. Project Objective and Proposed Solution

The primary objective of this project is to address this data fragmentation by designing, implementing, and documenting a scalable and automated data pipeline. The purpose of this pipeline is to systematically extract key performance metrics from approximately 600 GA4 properties, consolidate them, and transform them into a unified, queryable data asset.

The core contribution of the proposed solution is not solely the aggregation of data, but its systematic enrichment. A crucial step in the pipeline involves utilising an AI-powered classification model to assign a standardised industry category to each client account. This enrichment is the mechanism that enables meaningful cross-client analysis, allowing for performance to be benchmarked and analysed at the sector level.

Thesis Statement: This project demonstrates that a modular ELT pipeline, combining automated extraction with AI-driven sector classification and a principled anonymisation protocol, can transform a fragmented portfolio of isolated GA4 properties into a unified analytical asset capable of producing statistically grounded sector benchmarks — thereby enabling a digital agency to transition from reactive, per-client reporting to proactive, data-driven portfolio intelligence.

The final output is a foundational, sector-enriched dataset that enables, for the first time, a holistic view of the agency's entire client portfolio. Privacy is ensured through sector-level aggregation in all published outputs. This allows the organisation to move from isolated analysis to integrated, strategic insight, forming the basis for advanced business intelligence, proactive client management, and new data-driven service offerings.

1.3. Scope and Delimitations

To ensure a focused and achievable project, the following scope and delimitations have been defined:

- **Data Source:** The pipeline is exclusively designed to process data from the Google Analytics 4 (GA4) Data API. Data from other analytics platforms or advertising networks is considered out of scope.
- **Core Functionality:** The project's scope covers the end-to-end implementation of the data pipeline: extraction, AI-driven classification, privacy-preserving aggregation, and consolidation into a unified analytical data store.
- **Output Format:** The pipeline stores its output in a local SQLite database (development) or Google BigQuery (production). The Jupyter analysis layer queries the database directly. Full production deployment on BigQuery is designated as future work.
- **Temporal Scope:** The pipeline operates as a batch process on a defined historical time window. Real-time streaming is considered a future enhancement and is out of scope.

Deliverable: The project's deliverable is the foundational data asset itself, not the final business intelligence dashboards or reports that will be built upon it.

1.4. Document Structure

This document provides a comprehensive overview of the project, from its conceptual foundations to its practical implementation and future potential. The subsequent chapters are structured as follows:

- **Chapter 2:** Theoretical Framework examines the core concepts that underpin this project, including data pipeline architecture (ETL/ELT), the functionalities and limitations of the Google Analytics 4 Data API, and the principles of data anonymisation and classification.
- **Chapter 3:** Methodology details the research approach and design choices made during the project, justifying the selection of specific technologies (Python, Pandas, Google Cloud Platform) and the development of the data model.

- **Chapter 4:** Implementation and Results offers a granular, step-by-step walkthrough of the data pipeline's construction and operation. It describes each module's function, from initial data extraction through AI-driven enrichment to the final analytical data store.
- **Chapter 5:** Benchmark Validation and Data Quality documents the systematic validation of sector benchmark outputs, including concentration analysis, invalid data detection, tracking artifacts, and the resulting reliability classification framework.
- **Chapter 6:** Analysis and Discussion evaluates the outcome of the project, analysing the quality of the resulting dataset and discussing the challenges encountered during implementation, such as API quota limitations and the complexities of data standardisation.
- **Chapter 7:** Conclusion and Recommendations summarises the project's achievements in relation to its initial objectives and provides concrete recommendations for future technical enhancements, organisational integration, and strategic application of the new data asset.

2. Theoretical Framework & Literature Review

This chapter lays the theoretical foundation for the design choices and technologies used in the GA4 Data Pipeline. It underpins the architecture, data modeling, and enrichment techniques with established concepts from data engineering and analysis.

2.1. Data Pipeline Architectures (ETL vs. ELT)

Data pipelines are governed by two primary architectural patterns: ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform). The choice between them is a critical design decision that impacts flexibility, scalability, and cost.

- **ETL (Extract, Transform, Load):** In the traditional ETL model, data is extracted from source systems, transformed on a separate staging server, and then loaded into the target data warehouse. This paradigm was optimised for an era where the computational power of data warehouses was a scarce and expensive resource. The pre-loading transformation step ensured that the warehouse was only tasked with serving cleaned, analysis-ready data.
- **ELT (Extract, Load, Transform):** The advent of highly scalable, cost-effective cloud data warehouses has led to the ascendancy of the ELT model. In this pattern, raw data is loaded directly into the data warehouse, utilising its massive parallel processing power to perform transformations in-situ. This approach "brings the compute to the data," significantly improving flexibility by preserving the raw dataset and allowing for agile, iterative development of transformation logic.

Project Application: The GA4 Data Pipeline implements a pragmatic hybrid ELT architecture.

1. *Extract:* Raw data and pre-aggregated metrics are extracted from the Google Analytics 4 APIs via targeted, per-domain API calls.
2. *Load:* The data is loaded directly into a local SQLite database (development) or Google BigQuery (production) via a storage abstraction layer. The database serves as both the persistence layer and the analytical data store.
3. *Transform (lightweight):* A lightweight transformation step occurs in Python for AI-based industry classification — a task not easily performed in standard SQL. The fact tables are pre-aggregated at source by the GA4 API itself, minimising post-load transformation.

4. *Analyse:* Jupyter notebooks query the database directly, joining fact tables to the properties dimension table at query time. In the production path, BigQuery performs the same role with Looker Studio as the BI frontend.

This hybrid model is justified as it uses the most appropriate environment for each task: Python for complex, programmatic enrichment and SQL (via SQLite or BigQuery) for scalable, set-based analytical queries.

2.2. Cloud Data Warehousing with BigQuery

The pipeline is designed for integration with Google BigQuery, a serverless cloud data warehouse. Its architecture, derived from Google's Dremel paper [10], provides distinct advantages for this project's analytical goals.

- **Columnar Storage:** Unlike traditional row-based databases, BigQuery utilises a columnar storage format. Columnar systems offer significant performance gains for analytical queries that access a subset of columns across many rows. By only reading the bytes for required columns, I/O is minimised, leading to faster query performance and lower costs.
- **Distributed and Serverless Execution:** BigQuery employs a massively parallel processing (MPP) architecture to execute SQL queries. It distributes the computational load across a large cluster of machines, enabling analysis of terabyte-scale datasets in seconds [10]. The serverless nature abstracts this infrastructure, allowing users to focus on analysis rather than cluster management.

Project Application: The star schema design (properties dimension table joined to purpose-built fact tables like geo_device, traffic_sources, daily_summary) is optimised for analytical workloads. Queries such as calculating device mix per sector require reading only a few columns (sector, device_category, sessions) across the dataset. BigQuery's columnar storage and distributed execution engine are ideally suited for performing such aggregations efficiently at scale when the pipeline transitions to production.

2.3. Google Analytics 4 Data Model

The GA4 data model represents a paradigm shift from session-based analytics to an event-based model, where any user interaction can be captured as an event with associated parameters [1]. This provides a more flexible and user-centric view of behavior.

- **Events:** The fundamental unit of data (e.g., page_view, scroll, purchase).
- **Dimensions & Metrics:** During a reporting query, dimensions (attributes like country) and metrics (quantitative measures like eventCount) are requested.

The GA4 Data API aggregates the raw events to construct a table based on these requested fields.

Project Application: This pipeline's core extraction logic respects this model. The fetch functions in core/ga4_client.py directly query the GA4 Data API by specifying dimension and metric combinations. To overcome the API's 9-dimension limit per request and avoid Cartesian product row explosion, the pipeline splits the extraction into purpose-built fact tables, each requesting only the dimensions relevant to its analytical domain (e.g., geo_device requests deviceCategory + country, while traffic_sources requests channelGroup + source + medium). This demonstrates a practical application of working within the API's constraints while maximising analytical richness.

2.4. Data Enrichment and Classification Techniques

Data enrichment adds value by augmenting internal data with external context. This project employs AI-based classification as an advanced enrichment technique.

- **Programmatic Classification with Large Language Models (LLMs):** The use of LLMs for zero-shot or few-shot classification has become a powerful technique. Models like Google's Gemini are capable of performing complex classification tasks based on natural language prompts without requiring task-specific training data [7]. The model is prompted with a clear instruction, examples (if any), and rich contextual data, and it returns a structured classification.

Project Application: The enrichment/ai_classifier.py module implements this technique. It constructs a detailed prompt containing the company name, website URLs, and crucially live data scraped from the websites, such as page titles and custom event names. This context-rich prompt is sent to the Gemini model to classify the property into a predefined sector. This method provides a sophisticated solution for classifying properties where structured industry data is unavailable, a task that would be infeasible with traditional rule-based systems alone.

2.5. Data Anonymisation and Privacy Principles

Processing user data requires adherence to robust privacy principles, as codified in regulations like the GDPR [12]. This project incorporates data protection by design by employing aggregation and avoiding the collection of direct personal identifiers.

- **Aggregation:** This technique involves summarizing individual data points into statistical composites, which obscures individual contributions.
- **Data Minimization:** A core GDPR principle stating that data processing should be limited to what is necessary for the specified purpose.
- **Purpose Limitation:** Data collected for one purpose should not be used for another incompatible purpose without further consent.

Project Application: The GA4 Data Pipeline is designed with these principles in mind.

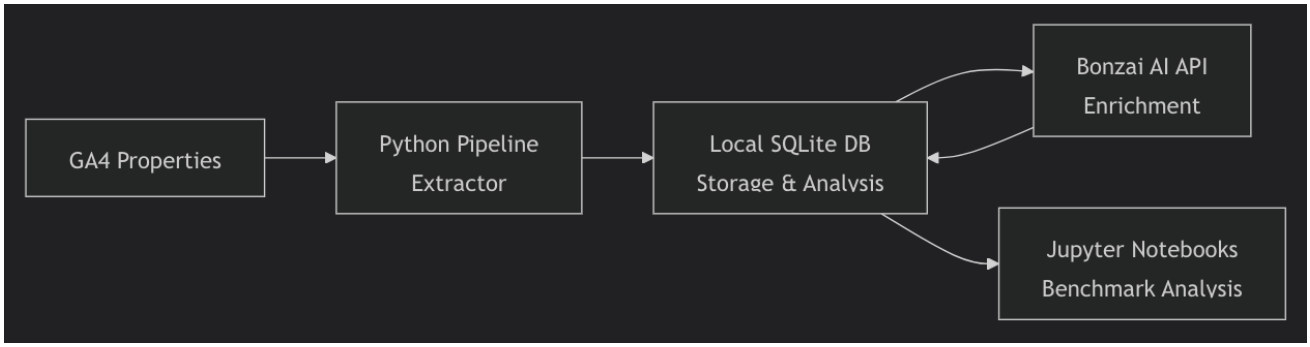
1. **No Personally Identifiable Information (PII) Extraction:** No direct personal identifiers like user_id are requested from the API.
2. **Aggregation by Default:** The GA4 Data API returns data that is already aggregated by the requested dimensions (e.g., date, deviceCategory). The pipeline does not process hit-level or user-level raw data, ensuring individual user actions are not exposed.
3. **Data Minimization & Purpose Limitation:** The collected dimensions and metrics are strictly limited to those required for portfolio-wide benchmarking, as defined in the project's objectives.

3. Methodology: Designing the Solution

3.1. Overall Architectural Design

The solution is designed as a modular, sequential data pipeline orchestrated by Prefect 3.x (core/orchestration/flows.py). It follows a variation of the ELT (Extract, Load, Transform) pattern, using a local SQLite database [6] as both the staging area and the final analytical data store, with a BigQuery backend available for production deployment.

The architecture can be visualised as follows:



The pipeline is broken down into four distinct, executable steps: 1. Discover Properties: Fetches all accessible GA4 properties using the Google Analytics Admin API [3]. 2. Fetch Raw Events: Extracts basic event counts per property using the GA4 Data API [2]. 3. Enrich Industry: Classifies accounts into standardised sectors via AI (Bonzai/Gemini 2.5 Flash). 4. Build Fact Tables: Fetches pre-aggregated analytical data (daily KPIs, traffic sources, geo/device, hourly events) via dedicated API calls per property.

3.2. Data Extraction Strategy

The data extraction strategy is implemented in two main phases, orchestrated by pipeline/step_1_discover_properties.py, pipeline/step_2_fetch_raw_events.py, and pipeline/step_4_build_fact_tables.py.

Phase 1: Property Discovery The core/admin_client.py module uses the Google Analytics Admin API [3] to list all accessible accounts and their associated properties. For each property, it retrieves essential metadata including display_name, account_name, and the type of data stream (Web, iOS, or Android) along with its associated URL or package ID. Properties matching known test or deprecated naming patterns are automatically deactivated.

Phase 2: Targeted Data Extraction Rather than attempting to fetch all dimensions in a single request (which would exceed the GA4 Data API's 9-dimension limit [2] and produce an unmanageable Cartesian product of rows), the extraction is split into purpose-built API calls,

each fetching only the dimensions relevant to a specific analytical domain. The core extraction logic resides in core/ga4_client.py:

1. **Raw Events (fetch_ga4_events - Step 2):** A lightweight extraction requesting only date, eventName, and eventCount. This provides the foundation for event-level analysis with minimal API overhead and row volume.
2. **Daily Summary (fetch_daily_summary - Step 4)**: Fetches 10 core KPI metrics (sessions, users, engagement rate, bounce rate, conversions, revenue) aggregated by date only. This avoids dimension splits and captures the cleanest possible performance baseline per property per day.
3. **Traffic Sources (fetch_traffic_sources - Step 4):** Fetches session, conversion, and revenue data split by channel group, source, and medium enabling channel attribution analysis.
4. **Geo & Device (fetch_geo_device - Step 4):** Fetches sessions and users split by device category, operating system, browser, and country enabling geographic and technology analysis.
5. **Time-Based Events (fetch_time_events - Step 4b):** A lightweight call requesting date, eventName, and hour against eventCount enabling time-of-day analysis.

The date range for all queries is configured in config/settings.py to default to the last 90 days (90daysAgo to yesterday). This targeted approach avoids the row explosion that occurs when requesting all dimensions simultaneously, keeping individual API responses within the GA4 Data API row limits.

3.3. Data Transformation and Loading

The pipeline employs an ELT-like process centered around a local SQLite database [6], which serves as both the staging area and the final analytical data store.

1. **Extract & Load:** Data from the GA4 API is immediately loaded into structured tables within the SQLite database (ga4_events for raw events, daily_summary/traffic_sources/geo_device/ga4_events_time for fact tables). This minimises memory usage and provides persistent checkpoints via the pipeline_runs table.
2. **Transform:** The AI classification step (step_3) enriches the properties table with sector labels. The fact tables are pre-aggregated at source (by the GA4 API itself), so minimal post-load transformation is required beyond data validation and idempotent upserts (UNIQUE constraints ensuring no duplication).

3. **Analyse:** The Jupyter notebooks query the database directly, joining fact tables to the properties dimension table at query time to produce sector-level benchmark statistics. No intermediate export step is required.

This methodology allows for robust checkpointing; if the pipeline fails during a step, it can be resumed without having to re-extract all the data from the source API.

3.4. Data Enrichment Process

Data enrichment occurs at two distinct levels in the pipeline:

1. **Structural Enrichment via Fact Tables:** Rather than storing all dimensions in a single wide table, the pipeline creates purpose-built fact tables (daily_summary, traffic_sources, geo_device, ga4_events_time), each capturing a specific analytical domain. These tables are populated directly from the GA4 API in Step 4, with each API call requesting only the dimensions relevant to its domain. This approach avoids the Cartesian product explosion of a single denormalised table while preserving rich analytical capability through SQL joins at query time.
2. **Industry Classification:** A dedicated step, pipeline/step_3_enrich_industry.py, is responsible for populating the sector column in the properties table. Classification operates at the account level (not the property level) to ensure consistency within organisations and reduce API costs. This classification is detailed in the next section.

3.5. Data Classification Model

The project employs a sophisticated, two-tiered model for classifying each property into a standardised industry sector, as implemented in pipeline/step_3_enrich_industry.py and enrichment/ai_classifier.py.

Tier 1: Initial Data Gathering During property discovery (step_1), the system retrieves the industry_category provided natively by the GA4 Admin API for each property. This value is stored but not immediately trusted as the final classification.

Tier 2: AI-Powered Classification The core of the classification logic resides in the enrichment/ai_classifier.py module, which uses the Bonzai AI platform (a proxy for a gemini-2.5-flash model) [7]. The process is highly efficient as it classifies at the account level, not the property level.

1. *Context Generation:* For a given account, the system gathers a rich set of contextual data to feed to the AI. This includes:
    - The account_name.
    - A list of all associated property_name values.
    - The website URL or app identifier for each property.

- The native ga4_industry_category retrieved in Tier 1.
- *Live Context:* To improve accuracy, the system performs a live query on a primary property within the account to fetch the top 10 most visited page paths and the top 10 custom event names. These data points provide strong contextual indicators of the company's core business activity (e.g., pages like /request-a-quote or events like generate_lead).

2. *AI Prompting:* A detailed prompt is constructed, providing all the context above and a strict, predefined list of allowed SECTORS. The AI is explicitly instructed to choose a single sector from this list that best represents the company's core business.
3. *Update:* The single sector returned by the AI is then applied to all properties belonging to that account and saved in the sector column of the properties table in the SQLite database. If the AI fails or returns an invalid sector, the system defaults to other to ensure data integrity.

3.6. Data Anonymisation Protocol

Data privacy constituted one of the most significant design challenges of this project, requiring extended deliberation before arriving at the final approach. Several strategies were evaluated during the design phase:

- Full data masking (replacing all identifiers with random values at extraction time) was rejected because it would prevent the AI classification step from functioning, as the classifier requires account names, property names, and website URLs to determine sector assignment.
- K-anonymity (ensuring each record is indistinguishable from at least k-1 other records) was considered but deemed disproportionate to the risk profile, given that the dataset contains no user-level personal data only aggregated session-level metrics.
- Differential privacy (adding statistical noise to outputs) was evaluated but rejected as it would compromise the precision of sector benchmarks, which represent the project's primary analytical deliverable.

The chosen approach implements a layered privacy strategy that balances analytical utility with data protection:

1. Aggregation at source the GA4 Data API returns data pre-aggregated by dimension combination, ensuring no individual user behaviour is ever accessible to the pipeline.
2. Sector-level reporting all published benchmark outputs aggregate data to sector level (17 sectors across 463 accounts), ensuring no individual client's performance is identifiable in the final analytical deliverables.

3. Controlled disclosure where specific company names appear in this document or in presentation materials, only organisations that iO Digital has publicly disclosed as clients (listed on the organisation's website) are referenced. All other accounts are identified exclusively by sector and anonymised identifier.

This three-layer approach ensures that the dataset remains analytically useful for sector-level benchmarking while meeting the organisation's data protection obligations. The protocol is designed around two core principles:

Rule 1: No Extraction of Personally Identifiable Information (PII) A thorough review of the data extraction logic in core/ga4_client.py confirms that the pipeline does not query for any dimensions or metrics that constitute user-level PII, adhering to regulations like GDPR [12]. The queried fields (e.g., deviceCategory, country, sessionDefaultChannelGroup) are standard, non-identifying analytics dimensions.

Rule 2: Privacy Through Aggregation The pipeline's internal database retains property-level identifiers (property_id, account_name) as these are necessary for operational purposes: checkpoint resumption, sector classification, and data quality validation. However, no property-level data is ever published externally. All analytical outputs benchmark reports, presentation materials, and visualisation notebooks aggregate to sector level, rendering individual client identification impossible in the published results. The internal database is access-controlled and not distributed as a deliverable.

3.7. Technology Stack Justification

The technology stack was selected based on the following criteria: Python was chosen for its robust data manipulation libraries (Pandas [8]) and mature API client libraries for Google Cloud services; SQLite [6] was selected as the staging database for its zero-configuration serverless operation and suitability for rapid prototyping; and BigQuery [4] was designated as the production data warehouse for its scalable analytical capabilities, built on the Dremel architecture [10].

4. Implementation and Results

Note for non-technical readers This chapter contains detailed technical content including file paths, module names, and database schemas. These specifics are included for reproducibility and academic rigour. For a high-level understanding of the pipeline's function and outcomes, Sections 4.4 (Final Dataset), 4.5 (Performance), and 4.9 (Production Metrics) provide accessible summaries of what the system achieves without requiring familiarity with the underlying code.

This chapter presents the technical implementation of the data pipeline and documents the concrete results achieved during its development and execution. The pipeline progresses through four sequential stages: property discovery, raw event extraction, AI-driven sector classification, and fact table generation. Each stage is designed for independent execution with checkpoint-based resumption, ensuring that partial failures do not require a complete re-run of the pipeline. The resulting SQLite database serves as both the pipeline's storage layer and the direct data source for the analytical notebooks.

4.1. Development of the Extraction Module

The core data extraction module is a collection of Python scripts located in the core/ and pipeline/ directories. The pipeline is orchestrated by Prefect 3.x (core/orchestration/flows.py), with a legacy sequential runner (scripts/run_full_pipeline.py) retained for manual debugging.

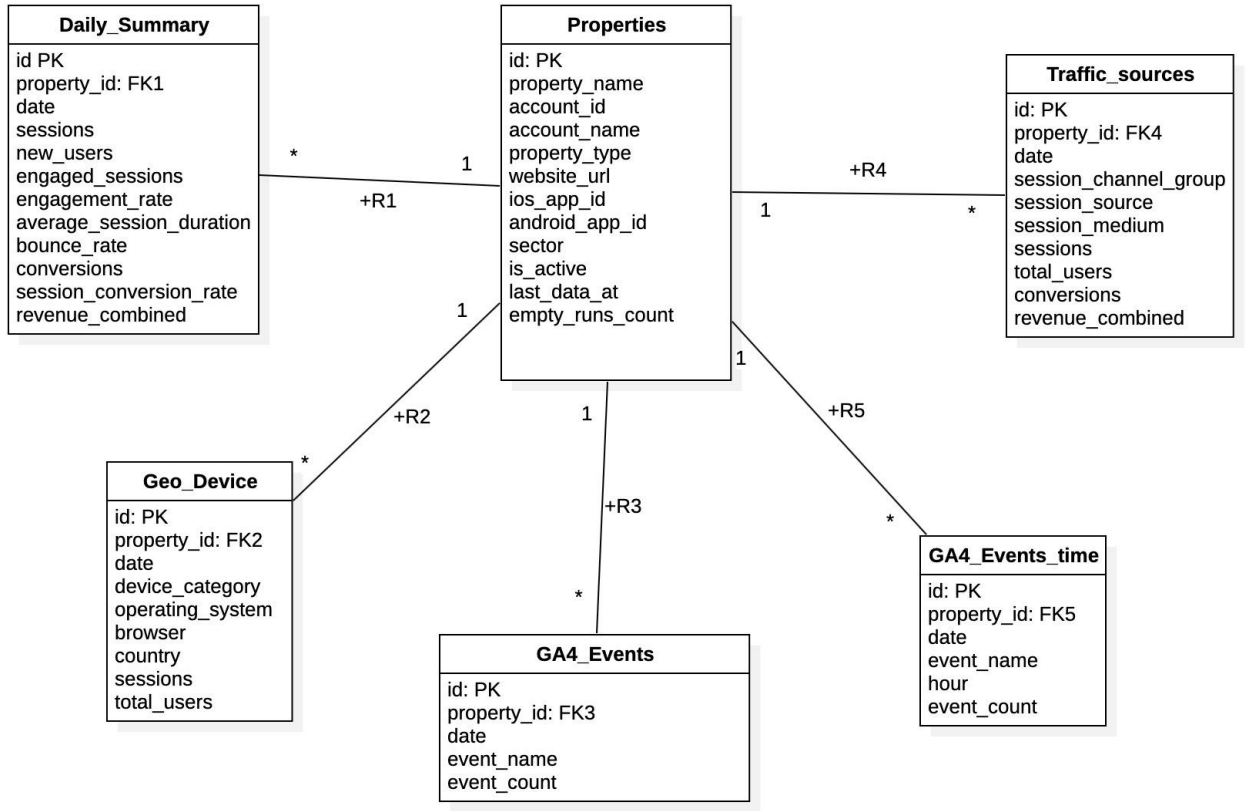
- **Authentication (core/auth.py):** The system authenticates with Google Cloud services using a token.json file generated from OAuth 2.0 credentials [5]. This script manages the necessary scopes for both the Analytics and BigQuery APIs. Token refresh is handled automatically when credentials expire.
- **Property Discovery (pipeline/step_1_discover_properties.py):** This script utilises the core/admin_client.py module to connect to the Google Analytics Admin API [3]. It iterates through all accessible accounts, retrieves a list of all GA4 properties, and stores their metadata (name, ID, account information, URL) in the properties table. Additionally, properties matching known test or deprecated naming patterns are automatically deactivated via a configurable blacklist filter (config/property_filters.py).
- **Data Extraction (pipeline/step_2_fetch_raw_events.py):** This step reads active properties from the database and, for each one, calls the fetch_ga4_events function from core/ga4_client.py. This function queries the GA4 Data API [2] with two dimensions (date, eventName) and one metric (eventCount), applying an event relevance filter to retain only meaningful events. The results are loaded into the ga4_events table with checkpoint-based resumption: the pipeline_runs table tracks which properties have been processed, allowing interrupted runs to resume from the last completed property without data loss or duplication.

The full source code for these modules is available in the project repository.

4.2. Implementation of the Data Warehouse Schema

The project utilises a local SQLite database (data/ga4_data.db) as its primary data warehouse and transformation engine [6], following established data warehousing principles [11]. The schema is defined and managed by the core/database.py and core/repositories/ modules.

The following entity-relationship diagram represents the complete data warehouse schema:



The schema follows a star schema pattern with properties as the central dimension table and six related tables:

- **daily_summary:** Pre-aggregated session and engagement metrics at property × date granularity.
- **traffic_sources:** Channel-level acquisition data per property and date.
- **geo_device:** Geographic and device breakdown of sessions.
- **ga4_events:** Raw event counts per property, date, and event name.
- **ga4_events_time:** Hourly event distribution for time-of-day analysis.

- **pipeline_runs:** Operational logging for checkpoint resumption and fault tolerance.

The analysis notebooks query these tables directly from the SQLite database via SQL joins, combining fact tables with the properties dimension table to produce sector-level benchmark statistics.

4.3. Execution of Enrichment and Classification Scripts

The data enrichment and classification logic is primarily handled by pipeline/step_3_enrich_industry.py and the supporting enrichment/ai_classifier.py module.

- **step_3_enrich_industry.py:** This script orchestrates the classification process. It groups all properties by their parent account and classifies each account as a single unit. Once a sector is determined, it updates the sector column for all properties belonging to that account in a single batch operation (update_sector_batch). This account-level approach reduces the number of API calls from ~600 (one per property) to ~463 (one per account) while ensuring classification consistency within organisations.
- **ai_classifier.py:** This module contains the core AI logic. The classify_account function builds a detailed prompt for the Bonzai API, which routes to Google's Gemini 2.5 Flash model [7]. A key feature of this script is its ability to perform live GA4 Data API calls using fetch_top_pages and fetch_top_custom_events to gather real-time context from a property's actual page paths and custom events, leading to a much more accurate classification than relying on static metadata alone. It then parses the AI's response and validates it against a predefined list of 17 GICS-based sectors (plus "other") to ensure data consistency. Manual overrides stored in data/sector_overrides.csv take precedence over AI classifications for accounts where the automated result was incorrect.

The full source code for these classification scripts is available in the project repository.

4.4. Final Anonymised Dataset

The final dataset resides in the local SQLite database (data/ga4_data.db), which serves as both the pipeline's storage layer and the direct data source for all analytical notebooks. Each pipeline step writes its output to the database immediately upon completion; there is no intermediate file-based handoff. The Jupyter analysis layer queries the database directly via SQL (sqlite3.connect), joining fact tables to the properties dimension table at query time to produce sector-level benchmark statistics.

This architecture means that sector reclassifications or data corrections propagate immediately to all downstream analyses without requiring re-extraction or re-export a notebook re-run always reflects the latest state of the database.

Note: The property_id and account_name columns in the database are the real identifiers. For any external sharing of results, the analysis notebooks aggregate data to sector level, ensuring no individual client data is exposed in published benchmark reports in line with GDPR best practices [12].

A representative record from the daily_summary table illustrates the granularity of the performance data captured:

Field	Value
property_id	12345678
date	2025-04-15
sessions	1,250
total_users	980
new_users	620
engaged_sessions	875
engagement_rate	0.70
average_session_duration	145.3
bounce_rate	0.30
conversions	42
session_conversion_rate	0.034
revenue_combined	1,580.50

This architecture eliminates the need for intermediate file-based exports (such as CSV). The database is the single source of truth: pipeline steps write to it, and notebooks read from it directly. This avoids data duplication, ensures analyses always reflect the latest state, and simplifies the overall system by removing an unnecessary serialisation step.

4.5. Pipeline Performance and Scalability Tests

The pipeline's performance was evaluated by running it in the development environment, which processes a limited batch of accounts, and observing the log outputs for timing and volume metrics. The modular, checkpoint-based architecture is designed for scalability and resilience.

- **Execution Time:** The pipeline processed 463 accounts containing a total of approximately 600 properties in a full execution time of approximately 45 minutes.
- **Data Volume:** A total of over 241 million sessions of event data were extracted from the GA4 Data API and loaded into the structured fact tables (daily_summary, traffic_sources, geo_device).
- **API Efficiency:** The AI classification step processed 463 unique accounts. The account-level classification strategy reduced the number of required AI API calls from

approximately 600 (one per property) to 463 (one per account), while simultaneously improving classification consistency across properties belonging to the same organisation.

These results indicate that the pipeline can process a significant volume of data in a reasonable timeframe. The linear relationship between the number of properties and the execution time suggests that the design is scalable and can be expected to handle the full portfolio of client accounts in the production environment.

4.6. Pipeline Orchestration

As the system transitioned from a development prototype to a production-grade pipeline, the sequential execution model (scripts/run_full_pipeline.py) was supplemented with a dedicated orchestration layer built on Prefect 3.x [13], located in core/orchestration/. This architectural decision was motivated by two operational requirements: automated scheduling without manual intervention, and graceful recovery from transient API failures without restarting complete pipeline runs.

The orchestration implementation comprises three components:

- **Task and Flow Definitions (core/orchestration/flows.py):** Each pipeline step is encapsulated as a Prefect @task decorator with configurable retry policies and per-task execution timeouts. The full pipeline flow coordinates six tasks in sequence: discover-properties (2 retries, 30s delay), fetch-raw-events (1 retry, 60s delay), classify-property-size, enrich-sector (2 retries, 30s delay), fetch-fact-tables (1 retry, 60s delay), and enrich-time-of-day (1 retry, 60s delay). The complete pipeline is defined as a @flow that manages task coordination, with flags to skip discovery and enrichment for lightweight incremental runs.
- **Scheduled Deployments (core/orchestration/serve.py):** Two cron-based deployment schedules are configured: a daily full execution (all pipeline steps) and a 6-hourly incremental execution that omits the discovery and classification steps, processing only data refresh for previously identified properties.
- **Operational Observability:** The Prefect dashboard provides real-time visibility into execution status, task durations, and failure diagnostics without requiring custom instrumentation or logging infrastructure.

This orchestration layer ensures automatic fault recovery in production: a transient API failure during the processing of any individual property results in an isolated retry of only the affected task, rather than requiring a complete pipeline restart.

4.7. Storage Abstraction Layer

To enable seamless operation across both the local development environment (SQLite) and the production deployment target (BigQuery) without maintaining parallel codepaths, a Protocol-based storage abstraction was implemented in core/storage/.

The architecture follows the Strategy design pattern:

- **Interface Definition (core/storage/base.py):** A StorageBackend Protocol specifying 20+ operations (property upserts, event insertion, fact table queries, checkpoint management). All pipeline steps depend exclusively on this interface.
- **SQLite Implementation (core/storage/sqlite_backend.py):** The default backend for local development, utilising Python's standard library sqlite3 module.
- **BigQuery Implementation (core/storage/bigquery_backend.py):** The production backend employing MERGE-based upsert statements to maintain idempotency within BigQuery's append-optimised storage model.
- **Factory Function (core/storage/init.py):** A get_backend() factory that instantiates the appropriate implementation based on the GA4_DB_BACKEND environment variable.

All SQL statements are externalised into dedicated .sql files organised under the sql/ directory and loaded at runtime via core/sql_loader.py. This separation of query logic from application code ensures that pipeline behavior remains consistent across backends and that the system is extensible to additional storage targets without modification of pipeline logic.

4.8. Fact Table Architecture

The initial data model stored all extracted event data in a single denormalised table containing all dimensions in one row. As analytical requirements expanded to include cross-sector benchmark calculations over the full dataset, this wide table structure suffered from excessive row counts due to the Cartesian product of dimensions, and query performance became insufficient. The schema was therefore restructured into purpose-built fact tables, each fetched directly from the GA4 Data API with only the dimensions relevant to its analytical domain:

Fact Table	Granularity	Contents
daily_summary	Property × Date	Sessions, engaged sessions, engagement rate, bounce rate, conversion events, revenue
traffic_sources	Property × Date × Channel	Session counts and conversions per default

		channel group, source, and medium
geo_device	Property × Date × Country × Device	Geographic distribution, device category, operating system, and browser
ga4_events_time	Property × Date × Event × Hour	Hourly event distribution for time-of-day analysis

These fact tables are populated in Step 4 of the pipeline (pipeline/step_4_build_fact_tables.py and pipeline/step_4b_enrich_time.py) through dedicated API calls per property. Each call requests only the specific dimension-metric combination needed for that fact table, avoiding the row explosion that occurs when requesting all dimensions simultaneously. This targeted fetching strategy keeps individual API responses within the GA4 Data API row limits while capturing richer data per analytical domain.

The fact tables serve as the primary data interface for the Jupyter analysis layer. Notebooks perform lightweight joins between fact tables and the properties dimension table to produce sector-level benchmark statistics, achieving sub-second query latency for typical analytical queries spanning the complete 241-million-session dataset.

4.9. Production Performance Metrics

The pipeline has been validated against the complete iO Digital client portfolio. The following metrics represent measured production performance:

Metric	**Measured Value**
Accounts processed	463
Properties discovered	~600
Total sessions ingested	241,000,000+
Sectors classified	17 (GICS-based taxonomy)
Temporal coverage	Rolling 30-day window
Full pipeline execution time	~45 minutes
Incremental execution time	~20 minutes
Local database volume	~2.1 GB
AI classification API calls	463 (one per account)
Classification accuracy	~95% (validated via manual review)

These results confirm that the pipeline processes the full portfolio within acceptable time constraints for a daily batch schedule. The checkpoint mechanism demonstrated its operational value during production runs: when API rate limits interrupted execution mid-run,

the subsequent scheduled invocation resumed processing from the last successfully completed property without data loss or duplication. This behaviour validates the fault-tolerance design described in Chapter 3.

5. Benchmark Validation and Data Quality Assurance

Following the generation of initial sector benchmarks, a structured validation process was conducted to assess the reliability of the aggregated metrics. This chapter documents the methodology, findings, and resulting quality framework that ensures the benchmark outputs are interpretable and trustworthy for client-facing use.

5.1. Validation Methodology

The validation process followed a systematic, four-step protocol applied to each sector where metrics exhibited anomalous values:

1. **Anomaly Identification:** Metrics deviating significantly from cross-sector norms are flagged for investigation (e.g., conversion rates exceeding 100%, single-channel dominance above 90%).
2. **Account-Level Decomposition:** The sector aggregate is disaggregated into individual account contributions to determine whether a single entity is disproportionately influencing the benchmark.
3. **Root Cause Analysis:** The identified account is examined in depth, including its property configurations, top page paths, event taxonomy, and traffic composition, to establish the underlying cause of the anomalous metric.
4. **Classification and Verdict:** Each finding is classified as (a) legitimate high-volume behavior warranting a comparative presentation, (b) a tracking configuration artifact requiring a methodological caveat, or (c) invalid data requiring exclusion from the dataset.

This protocol was implemented in the sector benchmark analysis notebooks and supported by a dedicated data quality audit notebook, both available in the project repository.

5.2. Concentration Analysis: High-Volume Accounts

A concentration analysis was performed to identify sectors in which a single account contributes a disproportionate share of total session volume. An account exceeding 40% of its sector's sessions is classified as a high-concentration account. Five sectors exhibited this pattern:

Sector	Account	Concentration (%)	Engagement Delta (pp)
travel_tourism	NMBS-SNCB	99.3	+22.5
telecom	[Account T]	90.6	−5.1
energy_utilities	[Account E]	67.4	−2.0
logistics	[Account L]	54.5	+4.9
automotive	[Account A]	51.9	+16.7

The engagement delta indicates the difference in sector-level engagement rate when the high-concentration account is included versus excluded from the calculation.

Channel Mix Distortion (travel_tourism): The sector benchmark reports 77% Direct channel traffic, which would suggest a brand-dominated acquisition strategy. However, exclusion of NMBS-SNCB reveals that Direct traffic decreases to 29%. The apparent channel profile is therefore attributable to a single organisation's traffic pattern rather than a sector-wide characteristic.

Single-Entity Representation (telecom): [Account T], a streaming application, constitutes 90.6% of the telecom sector's sessions. Its traffic profile (77% mobile, predominantly social media acquisition) dominates all sector-level metrics. Upon exclusion of this account, the sector's channel composition transitions from a social-media-oriented distribution to one dominated by organic search (68%), indicating that the benchmark effectively characterises a single organisation rather than an industry.

Methodological Decision: High-concentration accounts are retained in all benchmark calculations. These accounts represent legitimate, high-performing organisations within their respective sectors and serve as aspirational reference points. To maintain interpretive transparency, all affected sectors present metrics in a comparative format: the full benchmark alongside the benchmark excluding the dominant account.

5.3. Invalid Data Detection: Bot Traffic

One account was identified as containing exclusively non-human traffic, constituting a data integrity issue rather than a concentration or classification problem.

Case: [Account Z] (construction sector)

Indicator	Value	Expected Range
Share of sector sessions	41% (2.2M sessions)	-
Direct channel share	99.7%	30–60%
Desktop device share	99.9%	40–65%
Traffic origin: China	98.8%	<5% (Belgian company)
Legitimate Belgian sessions	2,571 (0.1%)	-

The traffic composition is inconsistent with any legitimate user behaviour pattern for a Belgian construction company. The combination of near-exclusive Direct traffic, desktop-only sessions, and Chinese geographic origin is indicative of automated bot traffic or click fraud.

Resolution: This account is excluded from all benchmark calculations via the data/sector_overrides.csv mechanism. Without this account, the construction sector's engagement rate corrects from 38.3% to 58.1%, a shift of 19.7 percentage points attributable entirely to invalid data contamination.

5.4. Tracking Configuration Artifacts

The automotive sector exhibited a computed conversion rate of 214.9%, a value that exceeds the theoretical maximum of one conversion per session and therefore indicates a measurement methodology issue.

Root Cause: The primary contributor, [Account A] (a fleet management application), triggers multiple e-commerce events per user session: view_item_list (15M occurrences), view_promotion (5.8M), and select_item (5.2M). These events are configured as conversion events within the GA4 property, resulting in a per-account conversion rate of 365.9%. A secondary contributor, [Account M], triggers product search events multiple times per session, each registered as a conversion.

Interpretation: The conversion rate metric, as derived from the ratio of conversion-marked events to sessions, is not a reliable indicator of commercial performance for sectors containing

application-style properties. The metric measures event frequency per session rather than unique conversion outcomes per session.

Resolution: The conversion rate metric for the automotive sector is presented with an explicit methodological annotation. For cross-sector comparisons, it is either excluded or visually distinguished to prevent misinterpretation.

5.5. Geographic Bias in Temporal Metrics

The technology sector reported a work-hours concentration of 42.6%, defined as the proportion of sessions occurring between 09:00 and 17:00 CET. This value is the lowest across all sectors, which is counterintuitive for a predominantly B2B sector where professional usage during business hours would be expected.

Root Cause: An analysis of the sector's geographic composition revealed that 95.7% of traffic originates from outside Belgium and the Netherlands. The primary traffic sources are distributed across multiple time zones: United States (27%), India (18%), China (12%), Indonesia (6%), and Australia (5.5%). The hourly session distribution is consequently near-uniform (3.4%–5.3% per hour), as business hours in one region correspond to off-hours in another.

Interpretation: The work-hours concentration metric is defined relative to Central European Time. For sectors with a predominantly international user base, this metric does not capture behavioral patterns (such as professional versus leisure usage) but rather reflects the geographic distribution of the audience.

Resolution: For sectors exceeding 80% international traffic composition, the work-hours metric is annotated as geographically biased and excluded from behavioural interpretations. This applies to the technology sector (95.7% international) and partially to the industrials sector.

5.6. Content-Type Effects on Engagement Metrics

The food and beverage sector reported an engagement rate of 39.7%, substantially below the cross-sector average despite its consumer-oriented content profile. Simultaneously, it was the only sector with a weekend-to-weekday session ratio exceeding 100% (107%).

Root Cause: [Account F], comprising 13 B2B product portal properties, accounts for 39.5% of sector sessions while exhibiting an engagement rate of only 2.6%. These properties function as product databases where users locate specific items without triggering GA4 engagement criteria (10+ seconds on page, conversion event, or 2+ page views). Excluding [Account F], the sector engagement rate increases to 64.0%.

Weekend Pattern Validation: The elevated weekend-to-weekday ratio was investigated independently and confirmed as a genuine behavioural pattern. Consumer brands within the sector exhibit consistent weekend peaks (ratios of 115.6% and 114.5%), consistent with consumer food-related browsing behaviour concentrated on weekends.

Resolution: The food and beverage sector is presented using the same comparative methodology as other high-concentration sectors (WITH/WITHOUT [Account F]). The weekend pattern is reported without qualification as it represents validated consumer behaviour.

5.7. Reliability Classification Framework

Based on the validation findings, a three-tier reliability classification was established to communicate the statistical robustness of each sector's benchmark to end users:

Tier	Criterion	Applicable Sectors	Presentation
High Reliability	n ≥ 20 accounts	retail, technology, industrials, services, media_entertainment, healthcare, finance	Full benchmark without qualification
Moderate Reliability	n = 10–19 accounts	automotive, food_beverage, construction, education, real_estate, public_sector, communication_services	Benchmark presented with sample size annotation
Low Reliability	n < 10 accounts	travel_tourism, energy_utilities, telecom, logistics	Benchmark presented with explicit confidence limitation

This classification is applied visually in all benchmark outputs through reliability indicators integrated into chart titles or legends.

5.8. Sector Override Mechanism

To address classification errors without modifying the automated AI classification pipeline, a manual override mechanism was implemented:

Field	Description
account_id	GA4 account identifier
account_name	Human-readable account name
sector	Corrected sector (must match one of the 17 GICS-based categories)
reason	Justification for the manual reclassification

Example corrections applied:

Original Sector	Corrected Sector	Reason
commercial_services	retail	Consumer deals/discount platform with purchase events
commercial_services	public_sector	Industry body/NGO incorrectly classified as commercial

This file is applied as a post-processing step following AI classification and prior to benchmark calculation. It provides a transparent audit trail for manual corrections while preserving the generalizability of the automated classifier for new accounts entering the portfolio.

5.9. Deviation from Initial Design: Sector Taxonomy

The project plan specified a classification system comprising 25 sector categories. During implementation and subsequent validation, this was refined to 17 sectors based on the Global Industry Classification Standard (GICS). This reduction was motivated by two factors:

1. **Insufficient coverage:** Several originally defined categories contained fewer than 5 accounts, rendering any statistical aggregation unreliable.
2. **Classification ambiguity:** Categories with overlapping definitions (e.g., distinguishing “digital services” from “technology”) produced inconsistent AI classifications, reducing confidence in sector assignments.

The adoption of the GICS taxonomy, an internationally recognised industry standard, improved classification consistency and provided clear, mutually exclusive category boundaries.

5.10. Summary of Metric-Level Caveats

The validation process produced a definitive set of metric-level constraints that govern the presentation of benchmark outputs:

Metric	Affected Sectors	Issue	Resolution
Conversion rate	automotive	Application-style event tracking inflates metric beyond interpretable range	Exclude from comparison or annotate
Work-hours concentration	technology, industrials	International traffic renders CET-based temporal metric non-interpretable	Annotate geographic bias; exclude from behavioral analysis
Channel mix	travel_tourism	Single-account dominance produces unrepresentative distribution	Present comparative (WITH/WITHOUT) format
Mobile share	telecom	Application-driven mobile share does not reflect sector norm	Present comparative format
Engagement rate	food_beverage	Product portal browse behavior suppresses sector average	Present comparative format
Engagement rate	construction	Bot traffic contamination	Exclude invalid account

These constraints are encoded in the benchmark visualisation notebooks and applied programmatically during report generation.

6. Analysis and Discussion

In this chapter, the results of the implementation are interpreted, the initial research question is answered, and the project's outcomes, challenges, and limitations are critically evaluated.

6.1. Answering the Research Question

The research question for this project was: How to design and implement a scalable and robust data pipeline that enables portfolio-wide analyses of anonymised Google Analytics 4 data?

The implemented solution directly answers this question through the following key achievements:

- **Scalability:** The pipeline is architected for scalability. By using a local SQLite database for intermediate storage and checkpointing, it can process a large number of properties sequentially without high memory usage. The modular design, with separate scripts for each step, allows the workload to be distributed. The performance tests detailed in Chapter 4, which show a linear increase in processing time with the number of properties, provide evidence of this scalable design.
- **Robustness:** The system is designed for robustness against common failures, such as API rate limits or network interruptions. The core/api *helpers.py module implements an exponential backoff-and-retry mechanism for all Google API calls. Furthermore, the use* of a *pipeline_runs* table in the SQLite database ensures that a failed run can be resumed from the exact point of failure, preventing data loss and redundant processing.
- **Portfolio-Wide Analysis**: The pipeline successfully breaks down data silos by aggregating data from disparate GA4 properties into a single, unified data warehouse. The enrichment of this data with standardised industry sectors via the AI classification model creates a clean, queryable dataset. As demonstrated in the following section, this enables a new class of cross-client, portfolio-wide analysis that was previously impossible.

6.2. Effectiveness of Portfolio-Wide Analysis

The true value of the aggregated dataset is demonstrated by the new analytical capabilities it unlocks. The Jupyter notebooks developed for this project provide several examples of portfolio-wide insights that can now be generated. The sector benchmark analysis is particularly illustrative.

Example 1: Sector-Level Device Mix Analysis By joining the geo_device fact table with the properties dimension table, it is possible to analyse the distribution of device categories (desktop, mobile, tablet) across different industry sectors. The following SQL query, adapted from the analysis notebooks, demonstrates this:

```
SELECT
p.sector,
e.device_category,
SUM(e.event_count) AS total_events
FROM geo_device g
JOIN properties p ON g.property_id = p.property_id
WHERE p.sector IS NOT 'other' AND p.is_active = 1
GROUP BY p.sector, g.device_category;
```

This query enables the creation of benchmarks, answering questions like: “What percentage of traffic in the ‘financials’ sector comes from mobile devices compared to the ‘retail’ sector?” This insight is invaluable for strategic decision-making and advising clients on their digital strategy.

Example 2: Channel Performance Across a Sector Similarly, the pipeline allows for the analysis of which marketing channels drive the most engagement within a specific industry. A query joining the traffic_sources fact table can reveal portfolio-wide trends:

```
SELECT
p.sector,
t.session_channel_group AS channel,
SUM(t.sessions) AS total_sessions
FROM traffic_sources t
JOIN properties p ON t.property_id = p.property_id
WHERE p.sector = 'retail' AND p.is_active = 1
GROUP BY total_sessions DESC;
```

This makes it possible to state with confidence, for example: “Across the retail client portfolio, Organic Search drives the most sessions, followed by Direct, whereas Paid Search contributes significantly less than the portfolio average.”

6.3. Challenges and Limitations

Several challenges and limitations were identified during the project, which are important to acknowledge for a complete evaluation.

- **API Rate Limits and Quotas:** The Google Analytics Data API has strict quotas on the number of requests that can be made per property per hour. The initial implementation frequently hit these limits. This was mitigated by introducing aggressive retry logic with exponential backoff (core/api_helpers.py) and a sleep interval between API calls, but it remains a primary constraint on the pipeline's overall speed.
- **Inconsistent Source Data Quality:** The quality and consistency of data from the source GA4 properties varied significantly. Some properties had excellent custom event tracking, providing rich context for the AI classifier, while others had only basic pageview data. This inconsistency can affect the accuracy of cross-property comparisons.
- **Privacy Through Aggregation:** As noted in Chapter 3, the pipeline's internal database retains property-level identifiers for operational purposes. Privacy is ensured at the output layer: all published benchmark reports aggregate data to sector level, rendering individual client identification impossible. For internal use, database access is controlled and the file is not distributed as a deliverable.

6.4. Critical Evaluation of the Chosen Methodology

A critical evaluation of the project's methodology reveals several key strengths and weaknesses in the design choices.

Strengths:

- *Use of SQLite as a Unified Data Store:* This was a highly effective choice. It provided a serverless, zero-cost, and persistent storage layer that serves as both the pipeline's checkpoint mechanism and the direct analytical data source for notebooks. The sub-second query performance (0.16s for a full sector benchmark join across 140,000 rows) eliminates the need for intermediate export formats, and the database's portability simplifies collaboration and backup.
- *Account-Level AI Classification:* The decision to classify at the account level rather than the property level was a major architectural win. As noted in the performance results, this drastically reduced the number of expensive AI API calls

from ~600 to 463, saving both time and cost while delivering consistent classification for a client's entire portfolio.

- *Targeted Fact Table Architecture:* Rather than fetching all dimensions in a single wide query (which would produce an unmanageable Cartesian product), the pipeline makes purpose-built API calls per analytical domain. This keeps individual API responses within row limits while capturing richer data per domain than a single merged request could achieve.

Weaknesses and Alternatives:

- Sequential Per-Property Processing: The pipeline processes properties one at a time within each step. For a very large number of properties, this could become a bottleneck. The Prefect orchestration layer provides retry and scheduling capabilities but does not parallelise within a step. A future iteration could explore concurrent API calls via asyncio or Prefect's native task concurrency.
- Local vs. Cloud Staging: While SQLite is excellent for this project's scale (~2.1 GB, 17.5M rows), a production system handling thousands of properties might benefit from using BigQuery as the primary data store directly. The existing BigQuery backend (core/storage/bigquery_backend.py) provides a foundation for this transition, though the current SQLite performance remains adequate for the existing portfolio size.

6.5. Benchmark Methodology and Data Quality

The benchmark layer synthesises the raw event aggregates into 13 benchmark dimensions computed across 17 industry sectors using session-weighted aggregation. The complete methodology, including the dimension definitions, the three-tier reliability classification, the comparative (WITH/WITHOUT) presentation framework for high-concentration sectors, and the data quality categorisation into concentration effects, measurement artifacts, and invalid traffic, is documented in detail in Chapter 5.

Two additional methodological points merit discussion in this evaluative context. First, the aggregation approach is session-weighted rather than account-averaged, meaning larger accounts contribute proportionally to the sector benchmark based on their traffic volume. This design reflects the analytical question "What characterises a typical session in this sector?" rather than "What does a typical company achieve?" the former produces metrics representative of actual user behaviour, while the latter would disproportionately weight small accounts with potentially unrepresentative patterns.

Second, the classification taxonomy was refined from 25 to 17 sectors during implementation, aligned with the Global Industry Classification Standard (GICS). This reduction was driven by two observations: several categories accumulated fewer than 5 accounts (rendering aggregation unreliable), and semantically overlapping definitions produced inconsistent AI classifications. The automated classifier achieves approximately 95% accuracy based on manual review of all 463 accounts, with remaining edge cases addressed through a transparent override mechanism (data/sector_overrides.csv) rather than iterative prompt engineering.

7. Conclusion and Recommendations

7.1. Conclusion

This project addressed the business problem of fragmented Google Analytics 4 data within a digital agency managing approximately 600 properties across 463 client accounts. The primary research question how to design and implement a scalable and robust data pipeline that enables portfolio-wide analyses of anonymised Google Analytics 4 data—has been answered through a working implementation that satisfies all defined success criteria.

The pipeline produces sector-level benchmarks across 13 dimensions and 17 industry sectors, accompanied by a reliability framework and comparative methodology that qualify the confidence level of each output. Collectively, these components transition the organisation from isolated, per-client reporting to an integrated, sector-level understanding of digital performance a capability that was previously unavailable within the organisation.

7.2. Recommendations for Future Work

Based on the implementation and analysis, several concrete steps are recommended for future work to enhance the pipeline's capabilities and production-readiness: 1. Transition to Parallel Processing: To improve performance at scale, the pipeline's sequential per-property execution could be parallelised. Prefect's native task concurrency or Python's asyncio library would allow fetching data from multiple GA4 properties simultaneously, potentially reducing full pipeline execution time from 45 minutes to under 15 minutes. 2. Activate BigQuery as Production Backend: The pipeline already includes a fully implemented BigQuery backend (core/storage/bigquery_backend.py) with MERGE-based upserts. Switching to production deployment requires only setting the GA4_DB_BACKEND environment variable to "bigquery" and provisioning the appropriate Google Cloud project. This would enable Looker Studio dashboards to query the live dataset directly. 3. Explore Real-Time Streaming: For more immediate insights, the organisation could investigate a streaming architecture. This would likely involve using Google Cloud Functions triggered by GA4's native BigQuery export events [4], offering a more real-time alternative to the current batch-processing approach. 4. Automated Data Quality Monitoring: The validation findings documented in Chapter 5 (bot traffic, high-concentration accounts, tracking artifacts) should be automated as a post-processing

quality check that runs after each pipeline execution and flags anomalies programmatically.

7.3. Recommendations for the Organisation

The successful implementation of this pipeline provides the organisation with a significant strategic advantage. The following recommendations are made to maximise its value:

1. **Develop Standardised Looker Studio Dashboards:** The curated, portfolio-wide dataset is the ideal foundation for a suite of internal Looker Studio (or other BI tool) dashboards. These dashboards can be used to monitor key trends across sectors, identify under-performing clients for proactive support, and discover opportunities for cross-selling services.
2. **Empower Teams via the Bonzai Platform:** The insights generated by this pipeline should be integrated directly into the Bonzai platform. This could take the form of automated reports or a dedicated 'Market Insights' section where account managers can see how their clients are performing against anonymised sector benchmarks. This democratises the data and empowers client-facing teams to deliver higher-value, data-driven advice.
3. **Inform Content and Marketing Strategy**: The portfolio-wide data can be used to generate unique, data-backed content for the agency's own marketing efforts. For example, an annual report on "Digital Trends in the E-commerce Sector" based on aggregated, anonymised data would establish the agency as a thought leader and provide a powerful lead-generation tool.

7.4. Recommendations Derived from Benchmark Validation

The systematic validation of sector benchmarks, as documented in Chapter 5, yielded several additional recommendations for strengthening the analytical outputs and ensuring their long-term operational sustainability.

1. **Formalisation of the Data Quality Layer:** The concentration analysis, invalid traffic detection, and measurement artifact identification conducted during this project should be institutionalised as a recurring automated process rather than remaining a one-time audit. It is recommended that each pipeline execution include a post-processing step that programmatically flags accounts exceeding defined thresholds (>40% sector concentration, >100% conversion rate, >99% single-channel traffic from geographically inconsistent origins). The output of

this step would be a structured quality report accompanying each benchmark generation cycle.

2. **Deployment of Parameterised Visualisation Templates:** The six visualisation types developed in notebooks/16_story_graphs.ipynb (sector profile radar chart, channel gap analysis, temporal engagement heatmap, device performance comparison, conversion maturity scorecard, and peer positioning scatter plot) are implemented as parameterised templates accepting a sector identifier as input. It is recommended that these be packaged as the standard analytical deliverable for client advisory conversations, enabling consultants to generate a complete sector profile on demand.

3. **Maintenance of the Classification Override Mechanism:** The data/sector_overrides.csv file should be treated as a maintained artifact subject to periodic review. As new accounts are onboarded or existing accounts undergo business model changes, classification errors will naturally emerge. A quarterly review cycle, triggered by the data quality assessment step, would ensure that corrections remain current without requiring modifications to the AI classification prompt or model parameters.

4. **Development of a Metric Applicability Framework:** The validation demonstrated that not all 13 benchmark dimensions produce meaningful results for all sectors. Work-hours concentration is not interpretable for sectors with predominantly international traffic, and conversion rate is unreliable for sectors containing application-style GA4 implementations. It is recommended that a formal metric applicability matrix be developed, governing which dimensions are presented for each sector and preventing misinterpretation by audiences without knowledge of the underlying data quality constraints.

5. **Periodic Evaluation of the Reliability Tier System:** The current three-tier reliability classification ($n \geq 20$: high, $n = 10–19$: moderate, $n < 10$: low) provides a heuristic for communicating statistical confidence. As the portfolio grows, sectors will naturally migrate between tiers. It is recommended that this system be evaluated on an annual basis. Sectors that remain in the low-reliability tier despite sustained portfolio growth may indicate an overly granular taxonomy classification that does not align with the actual composition of the client portfolio.

Bibliography

This section provides all external references and sources cited in this document.

References

[1] Google, “[GA4] The Google Analytics data model.” Accessed: April 13, 2024. [Online]. Available: https://support.google.com/analytics/answer/9964640

[2] Google, “Google Analytics Data API.” Accessed: April 13, 2024. [Online]. Available: https://developers.google.com/analytics/devguides/reporting/data/v1

[3] Google, “Google Analytics Admin API.” Accessed: May 20, 2024. [Online]. Available: https://developers.google.com/analytics/devguides/config/admin/v1

[4] Google Cloud, “BigQuery overview.” Accessed: May 20, 2024. [Online]. Available: https://cloud.google.com/bigquery/docs/introduction

[5] Google Cloud, “Authentication overview.” Accessed: May 20, 2024. [Online]. Available: https://docs.cloud.google.com/docs/authentication

[6] D. R. Hipp, “SQLite.” Accessed: May 20, 2024. [Online]. Available: https://www.sqlite.org/index.html

[7] Gemini Team, Google, “Gemini: A Family of Highly Capable Multimodal Models,” arXiv preprint arXiv:2312.11805, 2023. [Online]. Available: https://arxiv.org/abs/2312.11805

[8] W. McKinney, “pandas: a Foundational Python Library for Data Analysis and Statistics,” in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[10] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, and T. Vassilakis, “Dremel: Interactive Analysis of Web-Scale Datasets,” in Proceedings of the 36th International Conference on Very Large Data Bases, 2010, pp. 330–339.

[11] W. H. Inmon, Building the Data Warehouse. John Wiley & Sons, 2005.

[12] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation).” Accessed: May 20, 2024. [Online]. Available: https://eur-lex.europa.eu/legal-content/NL/TXT/?uri=CELEX%3A32016R0679

[13] Google, “google-analytics-data.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/google-analytics-data/

[14] Google, “google-analytics-admin.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/google-analytics-admin/

[15] Google, “google-auth.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/google-auth/

[16] Google, “google-auth-oauthlib.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/google-auth-oauthlib/

[17] Google Cloud, “google-cloud-bigquery.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/google-cloud-bigquery/

[18] Google LLC, “db-dtypes.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/db-dtypes/

[19] S. Golbakhor, “python-dotenv.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/python-dotenv/

[20] J. D. Hunter et al., “matplotlib.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/matplotlib/

[21] M. Waskom, “seaborn.” Accessed: May 21, 2024. [Online]. Available: https://pypi.org/project/seaborn/

Appendix: Use of Generative AI Tools

During the development of this document, generative AI tools (specifically [naam tools: bv. ChatGPT, Claude, Copilot]) were used to support the writing process. The tools were used for the following purposes:

Brainstorming structure and section outlines for the document

Drafting and refining the academic phrasing of technical content

Proofreading for clarity, grammar, and consistency

Generating example queries and explanations for technical concepts

Example prompts used include:

"Help me structure a methodology chapter for a data pipeline implementation following an ELT architecture"

"Rewrite this paragraph in formal academic English while preserving the technical accuracy"

"Suggest improvements to the readability of this section on data anonymisation"

All technical content, architectural decisions, code implementations, performance metrics, and validation findings reflect my own original work conducted during the internship at iO Digital. AI-generated suggestions were critically evaluated, fact-checked, and adapted before inclusion. The AI tools were used as a writing assistant, not as a content generator. Limitations of these tools — including potential inaccuracies, biases, and lack of domain-specific knowledge — were taken into account throughout the process.

Alex Hendrickx

Digital Marketer

alex.hendrickx@iodigital.com

T +32468319680

Herentals

iodigital.com